Serial Roulette Controller (*Roulette Game*)
Laboratório de Informática e Computadores 2024 / 2025 verão
Autores: Margarita Sysoeva 51403 / Pilar Gonçalves 51806

O módulo *Serial Roulette Controller (SRC)* implementa a interface com mostrador da roleta *(Roulette Display)*, fazendo a receção em série da informação enviada pelo módulo de controlo e entregando-a posteriormente ao mostrador de pontuação, conforme representado na Figura 1.

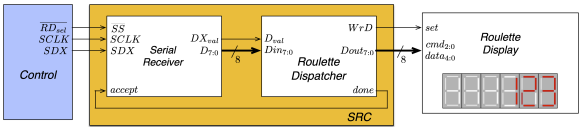

Figura 1: Diagrama de blocos do módulo *SRC*

Cmd			*data*					*Function*
2	*1*	*0*	*4*	*3*	*2*	*1*	*0*	
0	0	0	d_4	d_3	d_2	d_1	d_0	*update digit 0*
0	0	1	d_4	d_3	d_2	d_1	d_0	*update digit 1*
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
1	0	1	d_4	d_3	d_2	d_1	d_0	*update digit 5*
1	1	0	–	–	–	–	–	*update display*
1	1	1	–	–	–	–	0	*display on*
1	1	1	–	–	–	–	1	*display off*

Table 1: Operações no *Roulette Display*

O módulo *SRC* recebe em série uma mensagem constituída por oito (8) bits de informação e um (1) bit de paridade ímpar. A comunicação com este módulo realiza-se segundo o protocolo ilustrado na Figura 2. Os três (3) primeiros bits de informação, indicam o comando a realizar no mostrador da roleta, segundo a Tabela 1. Os restantes cinco (5) bits identificam o campo de dados. Tal como acontece com o *SLCDC*, o canal de receção série pode ser libertado após a receção da trama recebida pelo RouletteDisplay, não sendo necessário esperar pela sua execução do comando correspondente. Assim, o bloco *Roulette Dispatcher* pode ativar, prontamente, o sinal *done* para informar o bloco *Serial Receiver* que a trama já foi processada.

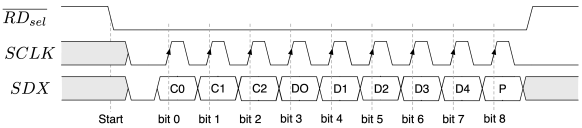

Figura 2: Protocolo de comunicação com o módulo *SRC*

1 Serial Receiver

O bloco *Serial Receiver* do módulo *SRC* foi implementado adotando, com as devidas adaptações, uma arquitetura similar à do bloco *Serial Receiver* do módulo *SLCDC*, neste caso adaptando a estrutura para a receção de oito (8) bits de informação em vez de cinco (5).

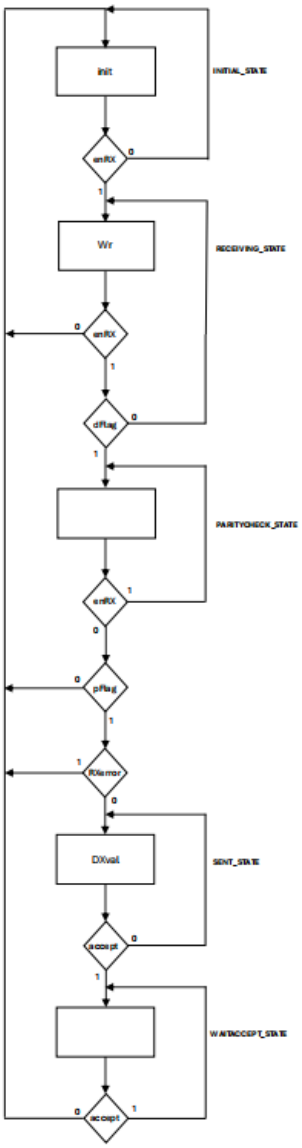

Figura 3: Máquina de estados do módulo *Serial Control*

O bloco *Serial Receiver* do módulo *SRC* foi desenvolvido com base numa máquina de estados descrita através de um ASM-chart, apresentada na Figura 3. Esta máquina de estados segue a mesma lógica de funcionamento utilizada no bloco *Serial Control* do módulo *SLCDC*, garantindo assim consistência no comportamento da receção serial entre ambos os módulos.

2 Roulette Dispatcher

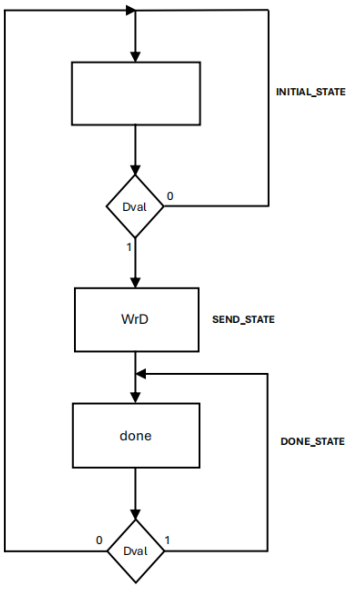

Figura 4: Máquina de estados do módulo *Roulette Dispatcher*

Após a receção de uma trama válida (proveniente do bloco *Serial Receiver*), o bloco *Roulette Dispatcher* inicia o processo de envio do comando recebido para o mostrador da roleta. O comportamento deste módulo é definido por uma máquina de estados sequencial (Figura 4), composta por três estados principais:

INITIAL_STATE: estado de espera, onde o módulo permanece enquanto *Dval* = '0', ou seja, enquanto não chega uma nova trama válida.

SEND_STATE: quando Dval = '1', o módulo avança para este estado e ativa o sinal *WrD*, indicando que o valor deve ser enviado para o mostrador.

DONE_STATE: após o envio, é ativado o sinal *done* e o módulo aguarda que *Dval* volte a '0' para retornar ao estado inicial.

Esta lógica garante que apenas tramas corretamente recebidas são processadas e enviadas uma única vez, evitando duplicações ou interpretações erradas no mostrador da roleta.

3 Control

Implementou-se o módulo *Control* em *software*, recorrendo a linguagem *Kotlin* e seguindo a arquitetura lógica apresentada na Figura 5.

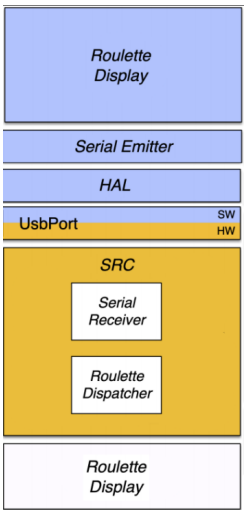

Figura 5: Diagrama lógico do módulo *Control* com a interface *Roulette Display*

3.1 Classe HAL

A classe HAL (*Hardware Abstraction Layer*) foi desenvolvida com o propósito de abstrair a comunicação direta com a porta USB da placa *d10lite*, facilitando a interação com os sinais digitais de entrada e saída do sistema. Esta camada de abstração permite ao programador manipular bits individuais ou conjuntos de bits de forma controlada e consistente, sem recorrer diretamente a operações de baixo nível.

Internamente, a classe mantém uma variável privada *currentState*, inicializada com o valor 0, que representa o estado atual dos bits de saída. Esta

variável é atualizada a cada operação de escrita, garantindo que a informação enviada para a porta reflete sempre o estado mais recente dos sinais.

As principais funções implementadas incluem: **init()**: Inicializa o HAL escrevendo o valor inicial (0x00) na porta, garantindo um estado limpo no começo; **isBit(mask: Int)**: Verifica se um determinado bit está ativo (a '1') na leitura da porta; **readBits(mask: Int)**: Retorna os bits de entrada correspondentes à máscara fornecida; **setBits(mask: Int)** e **clrBits(mask: Int)**: Permitem, respetivamente, ativar ou limpar bits específicos, atualizando o estado global do output; **writeBits(mask: Int, value: Int)**: Escreve valores específicos nos bits definidos pela máscara, garantindo que apenas esses bits são alterados.

3.2 Classe Serial Emitter

A classe *SerialEmitter* é responsável por enviar tramas de dados para os blocos físicos que implementam o *SerialReceiver*, como o *SLCDC* (para o LCD) e o SRC (para o display da roleta).

A função principal é **send(addr: Destination, data: Int, size: Int)**, que permite enviar uma trama bit a bit através de *SDX*, sincronizada com pulsos de relógio de *SCLK*. A trama é enviada segundo o protocolo definido no projeto, composta pelos bits de dados e um bit de paridade ímpar no final.

A comunicação inicia-se com a desativação da linha de seleção (*LCD_SEL_MASK* ou *RD_SEL_MASK*) de acordo com o destino (*Destination.LCD* ou *Destination.ROULETTE*). Em seguida, os bits do valor data são enviados da seguinte forma:

Cada bit é extraído e colocado em SDX através de *HAL.setBits(SDX_MASK)* ou *HAL.clrBits(SDX_MASK)*;

Um pulso é gerado no sinal SCLK para validar o bit (*setBits(SCLK_MASK*) seguido de *clrBits(SCLK_MASK)*);

Simultaneamente, é calculado o bit de paridade (*parity*) com *XOR* acumulativo dos bits enviados.

Após o envio dos bits de dados, é enviado o bit de paridade ímpar calculado:

Se *parity* == 0, o bit de paridade é '1', senão é '0', garantindo que o número total de bits a '1' na trama seja ímpar.

Finalmente, o destino é reativado com *HAL.setBits(destinationBit)* para concluir a transmissão.

Esta abordagem garante uma comunicação síncrona, controlada diretamente em software, sem necessidade de hardware adicional de transmissão, e foi projetada para ser compatível com qualquer *Serial Receiver* implementado em VHDL.

3.3 Classe Roulette Display

A classe *RouletteDisplay* controla o mostrador de seis dígitos responsável por apresentar os símbolos e números da roleta. O envio de dados é feito através de tramas seriais utilizando a classe *SerialEmitter*, de acordo com o protocolo implementado no *Serial Receiver* do módulo correspondente.

A função **init()** ativa o mostrador com os comandos **DISPLAY_ON** e **UPDATE_DISPLAY**, garantindo que o display está pronto a receber e apresentar informação.

A função principal **setValue(value: Int)** permite apresentar o mesmo símbolo ou dígito em todas as seis posições do mostrador. Antes de atualizar os dígitos, estes são limpos individualmente com um valor nulo (0x00), e só depois é enviado o novo valor. Cada envio inclui:

O valor a apresentar (codificado com **DATA_MASK**)

A posição no mostrador (codificada com **CMD_MASK**)

O método **setNumber(number: Int)** permite apresentar um número até 6 dígitos, sendo os dígitos alinhados à direita. Os dígitos são convertidos para inteiros, invertidos (de forma a serem apresentados da direita para a esquerda), e enviados individualmente para cada posição do mostrador.

A função **setValues(values: IntArray)** permite apresentar um valor diferente em cada uma das 6 posições. Primeiro, os dígitos são limpos e, de seguida, os novos valores são enviados para cada posição com os bits devidamente organizados.

A função **clear()** é um atalho para preencher o mostrador com espaços vazios (0x1F), utilizando *setValues*.

A função **animation()** realiza uma animação circular durante 5 segundos, alternando os símbolos em posições sequenciais do mostrador. O efeito simula o movimento da roleta, utilizando uma lista

fixa de valores que representam os símbolos da mesma.

Por fim, a função **off(value: Boolean)** permite desligar ou reativar o mostrador com base no envio dos comandos **DISPLAY_OFF** ou **DISPLAY_ON**, respeitando o protocolo de comunicação definido para o sistema físico.

4 Conclusões

O desenvolvimento do sistema de controlo da roleta permitiu integrar de forma eficaz os módulos desenvolvidos em VHDL com o software implementado em Kotlin.

Através do módulo *SRC*, foi possível receber em série os comandos provenientes do controlador, processá-los e encaminhá-los corretamente para o mostrador da roleta. A máquina de estados do *RouletteDispatcher* garantiu que cada comando fosse enviado apenas uma vez, evitando repetições ou leituras incorretas.

No lado do controlo, a utilização das classes *HAL*, *SerialEmitter* e *RouletteDisplay* permitiu modularizar e simplificar o envio das tramas, promovendo uma comunicação eficiente com os módulos. A camada HAL assegurou o acesso transparente à porta *UsbPort*, enquanto o *SerialEmitter* gerou tramas válidas segundo o protocolo definido. O *RouletteDisplay*, por sua vez, coordenou a exibição dos comandos recebidos, incluindo animações e apresentação de letras e números no display.

A arquitetura adotada facilitou os testes, a validação individual de cada módulo, e garantiu um sistema coerente e funcional.

A VHDL

A.1 SRC

```
library ieee;
use ieee.std_logic_1164.all;

entity SRC is
        port(
        SS: in std_logic;
        SDX: in std_logic;
        SCLK:in std_logic;
        MCLK: in std_logic;
        RESET: in std_logic;
        WrD:out std_logic;
        Dout: out std_logic_vector(7 downto 0)
        );
end SRC;

architecture structural of SRC is

component SerialReceiver_SRC is
        port(
        SS : in std_logic;
        MCLK : in std_logic;
        SDX : in std_logic;
        RESET : in std_logic;
        SCLK : in std_logic;
        accept : in std_logic;
        D : out std_logic_vector(7 downto 0);
        DXval : out std_logic
        );
end component;

component Roulette_Dispatcher is
        port(
        Dval : in std_logic;
        CLK : in std_logic;
        RESET : in std_logic;
        done : out std_logic;
        Din: in std_logic_vector(7 downto 0);
        Dout: out std_logic_vector(7 downto 0);
        WrD : out std_logic
        );
end component;

signal done_accept : std_logic;
signal Dsignal : std_logic_vector(7 downto 0);
signal Dval_signal : std_logic;

begin
U0: SerialReceiver_SRC port map ( MCLK => MCLK, RESET => RESET, SDX => SDX, SCLK => SCLK,
SS => SS, accept => done_accept, D => Dsignal, DXval => Dval_signal );

U1: Roulette_Dispatcher port map ( Dval => Dval_signal, Din => Dsignal, RESET => RESET,
CLK => MCLK, done => done_accept, WrD => WrD, Dout => Dout );

end structural;
```

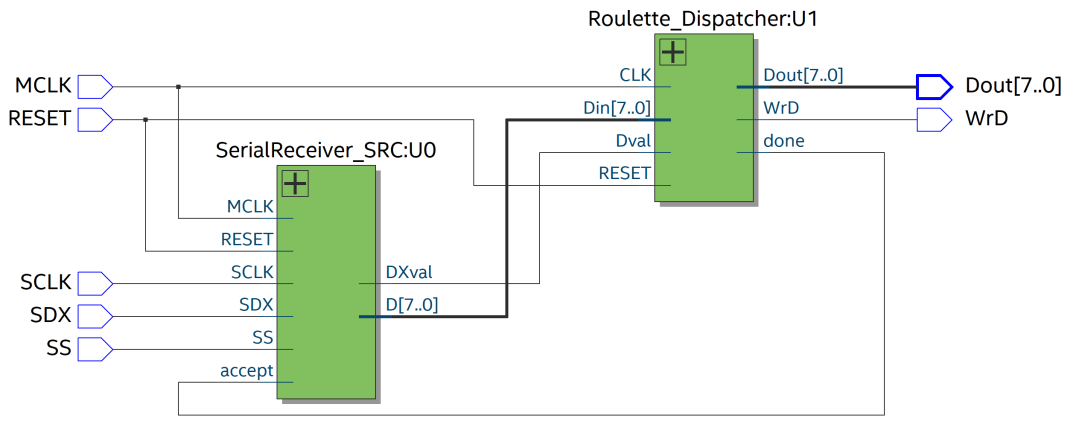

Figura 6: Netlist Viewer do *SRC*

A.2 Roulette_Dispatcher

```
library ieee;
use ieee.std_logic_1164.all;

entity Roulette_Dispatcher is
        port(
        Dval : in std_logic;
        CLK : in std_logic;
        RESET : in std_logic;
        done : out std_logic;
        WrD : out std_logic;
        Din : in std_logic_vector(7 downto 0);
        Dout : out std_logic_vector(7 downto 0)
        );
end Roulette_Dispatcher;

architecture behavioral of Roulette_Dispatcher is

type STATE_TYPE is (INITIAL_STATE, SEND_STATE, DONE_STATE);

signal Current_State, Next_State : STATE_TYPE;

begin

Current_State <= INITIAL_STATE when RESET = '1' else Next_State when rising_edge(clk);

GenerateNextState:
process (Current_State, Dval)
        begin
                case Current_State is
                        when INITIAL_STATE => if (Dval = '1') then
                                                Next_State <= SEND_STATE;
                                            else
                                              Next_State <= INITIAL_STATE;
                                            end if;
                         when SEND_STATE => Next_State <= DONE_STATE;


                         when DONE_STATE => if (Dval = '1') then
                                              Next_State <= DONE_STATE;
```

```
                                        else
                                            Next_State <= INITIAL_STATE;
                                        end if;

        end case;
end process;

WrD <= '1' when ( Current_State = SEND_STATE ) else '0';
done <= '1' when ( Current_State = DONE_STATE ) else '0';

Dout <= Din;

end behavioral;
```

A.3 SerialReceiver_SRC

```
library ieee;
use ieee.std_logic_1164.all;

entity SerialReceiver_SRC is
        port(
        SS : in std_logic;
        MCLK : in std_logic;
        SDX : in std_logic;
        RESET : in std_logic;
        SCLK : in std_logic;
        accept : in std_logic;
        D : out std_logic_vector(7 downto 0);
        DXval : out std_logic
        );
end SerialReceiver_SRC;

architecture structural of SerialReceiver_SRC is

component SerialControl_SRC is
        port(
        RESET : in std_logic;
        CLK : in std_logic;
        enRx : in std_logic;
        accept : in std_logic;
   pFlag : in std_logic;
   dFlag : in std_logic;
        RXerror : in std_logic;
   wr : out std_logic;
        init : out std_logic;
        DXval : out std_logic
        );
end component;

component ParityCheck_SRC is
        port (
        data: in std_logic;
        init: in std_logic;
        CLK: in std_logic;
        err: out std_logic
        );
end component;

component ShiftRegister_SRC is
        port(
        data: in std_logic;
        enableShift: in std_logic;
```

```
        CLK: in std_logic;
        D: out std_logic_vector(7 downto 0)
        );
end component;

component counterSR_SRC is
        port (
        CLK, clr: in std_logic;
        Q: out std_logic_vector (3 downto 0)
        );
end component;

component comparatorSR_SRC is
        port (
        A: in std_logic_vector (3 downto 0);
        B: in std_logic_vector (3 downto 0);
        TC: out std_logic
        );
end component;

signal SR_exit : std_logic_vector(7 downto 0);
signal wr_eS : std_logic;
signal counter_exit : std_logic_vector(3 downto 0);
signal init_exit : std_logic;
signal DXval_exit : std_logic;
signal err_RX : std_logic;
signal cinco_dF : std_logic;
signal seis_pF : std_logic;

begin
U0: SerialControl_SRC port map ( RESET => RESET, CLK => MCLK, enRx => not SS, accept => accept,
pFlag => seis_pF, dFlag => cinco_dF, RXerror => err_RX, wr => wr_eS, init => init_exit,
DXval => DXval_exit );

U1: ParityCheck_SRC port map ( data => SDX, init => init_exit, CLK => SCLK, err => err_RX );

U2: ShiftRegister_SRC port map ( data => SDX, enableShift => wr_eS, CLK => SCLK, D => SR_exit );

U3: counterSR_SRC port map ( CLK => SCLK, clr => init_exit, Q => counter_exit);

U4: comparatorSR_SRC port map ( A => counter_exit, B => "1000", TC => cinco_dF );

U5: comparatorSR_SRC port map ( A => counter_exit, B => "1001", TC => seis_pF );

DXval <= DXval_exit;
D <= SR_exit;

end structural;
```

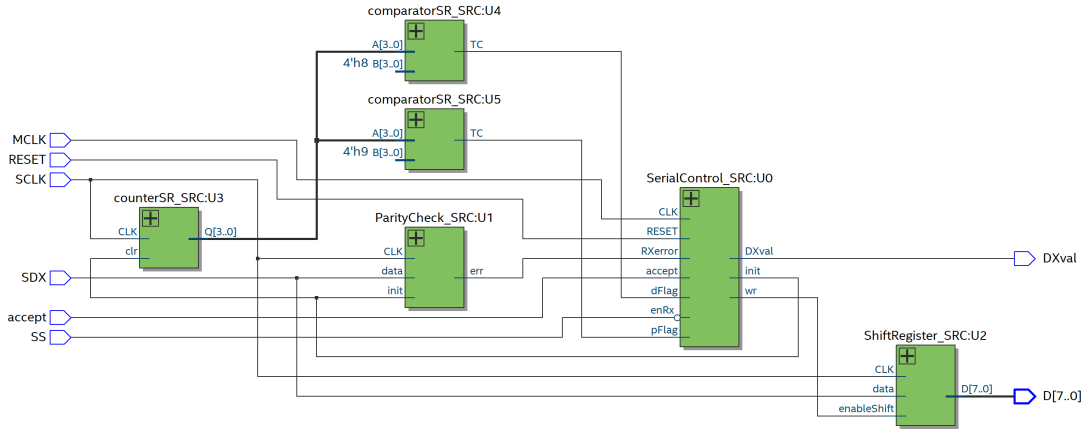

Figura 7: Netlist Viewer do *SerialReceiver_SRC*

A.4 SerialControl_SRC

```
library ieee;
use ieee.std_logic_1164.all;

entity SerialControl_SRC is
        port(
        RESET : in std_logic;
        CLK : in std_logic;
        enRx : in std_logic;
        accept : in std_logic;
   pFlag : in std_logic;
   dFlag : in std_logic;
        RXerror : in std_logic;
   wr : out std_logic;
        init : out std_logic;
        DXval : out std_logic
        );
end SerialControl_SRC;

architecture behavioral of SerialControl_SRC is

type STATE_TYPE is (INITIAL_STATE, RECEIVING_STATE, PARITYCHECK_STATE, SENT_STATE, WAITACCEPT_STATE);

signal CurrentState, NextState : STATE_TYPE;

begin

CurrentState <= INITIAL_STATE when RESET = '1' else NextState when rising_edge(CLK);

GenerateNextState:
process (CurrentState, enRX , accept , pFlag , dFlag , RXerror)
        begin
                case CurrentState is
                        when INITIAL_STATE        => if (enRX = '1') then
                                                        NextState <= RECEIVING_STATE;
                                                  else
                                                        NextState <= INITIAL_STATE;
                                                  end if;

                        when RECEIVING_STATE => if (enRX = '1' and dFlag = '1') then
```

```
                                                        NextState <= PARITYCHECK_STATE;
                                                elsif (enRX = '0') then
                                                        NextState <= INITIAL_STATE;
                                                else
                                                        NextState <= RECEIVING_STATE;
                                                end if;

                        when PARITYCHECK_STATE        => if (enRX = '0' and pFlag = '1'and RXerror = '0') then
                                                        NextState <= SENT_STATE;
                                                   elsif (enRX = '1') then
                                                        NextState <= PARITYCHECK_STATE;
                                                   else
                                                        NextState <= INITIAL_STATE;
                                                   end if;

                        when SENT_STATE        => if (accept = '1') then
                                                NextState <= WAITACCEPT_STATE;
                                           else
                                                NextState <= SENT_STATE;
                                           end if;

                        when WAITACCEPT_STATE        => if (accept = '0') then
                                                        NextState <= INITIAL_STATE;
                                                   else
                                                        NextState <= WAITACCEPT_STATE;
                                                   end if;
        end case;
end process;

init <= '1' when (CurrentState = INITIAL_STATE) else '0';
wr <= '1' when (CurrentState = RECEIVING_STATE) else '0';
DXval <= '1' when (CurrentState = SENT_STATE) else '0';

end behavioral;
```

A.5 ParityCheck_SRC

```
library ieee;
use ieee.std_logic_1164.all;

entity ParityCheck_SRC is
        port (
        data: in std_logic;
        init: in std_logic;
        CLK: in std_logic;
        err: out std_logic
        );
end ParityCheck_SRC;

architecture structural of ParityCheck_SRC is

component counterPC_SRC is
        port (
        CE, CLK, clr: in std_logic;
        Q: out std_logic_vector (3 downto 0)
        );
end component;

signal q: std_logic_vector(3 downto 0);

begin
```

```
U1: counterPC_SRC port map (CE => data ,CLK => CLK, clr => init, Q => q);

err <= not q(0);

end structural;
```

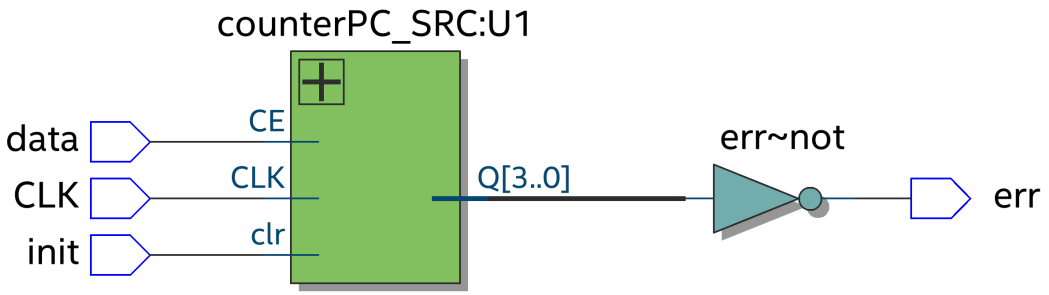

Figura 8: Netlist Viewer do *ParityCheck_SRC*

A.6 counterPC_SRC

```
library ieee;
use ieee.std_logic_1164.all;

entity counterPC_SRC is
        port (
        CE, CLK, clr: in std_logic;
        Q: out std_logic_vector (3 downto 0)
        );
end counterPC_SRC;

architecture structural of counterPC_SRC is

component adderPC is
        port(
   A, B: in std_logic_vector(3 downto 0);
   Ci: in std_logic;
   Co: out std_logic;
   S: out std_logic_vector(3 downto 0)
   );
end component;

component regPC is
        port(
        CLK : in std_logic;
        RESET : in std_logic;
        D : in std_logic_vector(3 downto 0);
        EN : in std_logic;
        Q : out std_logic_vector(3 downto 0)
        );
end component;

signal Adder_Reg: std_logic_vector (3 downto 0);
signal Reg_exit: std_logic_vector (3 downto 0);

begin

U0: adderPC port map (A => Reg_exit, B => "0001", Ci => '0', Co => open, S => Adder_Reg);
```

```
U1: regPC port map (CLK => CLK, RESET => clr, D => Adder_Reg, EN => CE, Q => Reg_exit);
Q <= Reg_exit;

end structural;
```

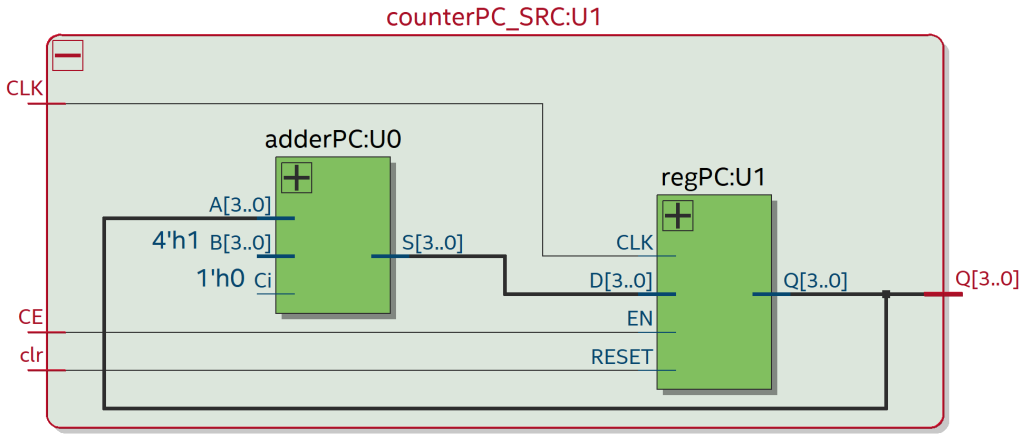

Figura 9: Netlist Viewer do *counterPC_SRC*

A.7 adderPC

```
library ieee;
use ieee.std_logic_1164.all;

entity adderPC is
        port(
        A, B: in std_logic_vector(3 downto 0);
        Ci: in std_logic;
        S: out std_logic_vector(3 downto 0);
        Co: out std_logic
        );
end adderPC;

architecture structural of adderPC is

component faPC is
        port(
        A, B, Ci : in std_logic;
        S, Co: out std_logic
        );
end component;

signal carry: std_logic_vector(3 downto 1);

begin

U0: faPC port map (A => A(0), B => B(0), Ci => Ci, S => S(0), Co => carry(1));
U1: faPC port map (A => A(1), B => B(1), Ci => carry(1), S => S(1), Co => carry(2));
U2: faPC port map (A => A(2), B => B(2), Ci => carry(2), S => S(2), Co => carry(3));
```

```
U3: faPC port map (A => A(3), B => B(3), Ci => carry(3), S => S(3), Co => Co);

end structural;
```

A.8 faPC

```
library ieee;
use ieee.std_logic_1164.all;

entity faPC is
        port(
   A, B, Ci: in std_logic;
   S, Co: out std_logic
   );
end faPC;

architecture structural of faPC is

component haPC is
    port(
    A, B: in std_logic;
    S, Co: out std_logic
    );
end component;

signal addAB: std_logic;
signal carry: std_logic_vector(1 downto 0);

begin

U1: haPC port map (A => A, B => B, S => addAB, Co => carry(0));
U2: haPC port map (A => addAB, B => Ci, S => S, Co => carry(1));

Co <= carry(0) or carry(1);


end structural;
```

A.9 haPC

```
library ieee;
use ieee.std_logic_1164.all;

entity haPC is
        port(
        A, B: in std_logic;
        S, Co: out std_logic
        );
end haPC;

architecture structural of haPC is
begin

S <= A xor B;
Co <= A and B;

end structural;
```

A.10 regPC

```
library ieee;
use ieee.std_logic_1164.all;
```

```
entity regPC is
        port(
        CLK : in std_logic;
        RESET : in std_logic;
        D : in std_logic_vector(3 downto 0);
        EN : in std_logic;
        Q : out std_logic_vector(3 downto 0)
        );
end regPC;

architecture structural of regPC is

component ffdPC is
        port(
        CLK: in std_logic;
        RESET: in std_logic;
        SET: in std_logic;
        D: in std_logic;
        EN: in std_logic;
        Q: out std_logic
        );
end component;

begin

U0: ffdPC port map(CLK => CLK, RESET => RESET, SET => '0', D => D(0), EN => EN, Q => Q(0));
U1: ffdPC port map(CLK => CLK, RESET => RESET, SET => '0', D => D(1), EN => EN, Q => Q(1));
U2: ffdPC port map(CLK => CLK, RESET => RESET, SET => '0', D => D(2), EN => EN, Q => Q(2));
U3: ffdPC port map(CLK => CLK, RESET => RESET, SET => '0', D => D(3), EN => EN, Q => Q(3));

end structural;
```

A.11 ffdPC

```
library ieee;
use ieee.std_logic_1164.all;

entity ffdPC is
        port(
        CLK : in std_logic;
        RESET : in std_logic;
        SET : in std_logic;
        D : in std_logic;
        EN: in std_logic;
        Q : out std_logic
        );
end ffdPC;

architecture structural of ffdPC is

begin


Q <= '0' when RESET = '1' else '1' when SET = '1' else D when rising_edge(CLK) and EN = '1';

end structural;
```

A.12 ShiftRegister_SRC

```
library ieee;
use ieee.std_logic_1164.all;
```

```
entity ShiftRegister_SRC is
        port(
        data: in std_logic;
        enableShift: in std_logic;
        CLK: in std_logic;
        D: out std_logic_vector(7 downto 0)
        );
end ShiftRegister_SRC;

architecture structural of ShiftRegister_SRC is

component ffdSR is
        port(
        CLK : in std_logic;
        RESET : in std_logic;
        SET : in std_logic;
        D : in std_logic;
        EN: in std_logic;
        Q : out std_logic
        );
end component;

signal exitFFD: std_logic_vector(7 downto 0);

begin

U1: ffdSR port map (CLK => CLK, RESET => '0', SET => '0', D => data, EN => enableShift, Q => exitFFD(0));
U2: ffdSR port map (CLK => CLK, RESET => '0', SET => '0', D => exitFFD(0), EN => enableShift, Q => exitFFD(1));
U3: ffdSR port map (CLK => CLK, RESET => '0', SET => '0', D => exitFFD(1), EN => enableShift, Q => exitFFD(2));
U4: ffdSR port map (CLK => CLK, RESET => '0', SET => '0', D => exitFFD(2), EN => enableShift, Q => exitFFD(3));
U5: ffdSR port map (CLK => CLK, RESET => '0', SET => '0', D => exitFFD(3), EN => enableShift, Q => exitFFD(4));
U6: ffdSR port map (CLK => CLK, RESET => '0', SET => '0', D => exitFFD(4), EN => enableShift, Q => exitFFD(5));
U7: ffdSR port map (CLK => CLK, RESET => '0', SET => '0', D => exitFFD(5), EN => enableShift, Q => exitFFD(6));
U8: ffdSR port map (CLK => CLK, RESET => '0', SET => '0', D => exitFFD(6), EN => enableShift, Q => exitFFD(7));

D(0) <= exitFFD(7);
D(1) <= exitFFD(6);
D(2) <= exitFFD(5);
D(3) <= exitFFD(4);
D(4) <= exitFFD(3);
D(5) <= exitFFD(2);
D(6) <= exitFFD(1);
D(7) <= exitFFD(0);

end structural;
```

A.13 ffdSR

```
library ieee;
use ieee.std_logic_1164.all;

entity ffdSR is
        port(
        CLK : in std_logic;
        RESET : in std_logic;
        SET : in std_logic;
        D : in std_logic;
        EN: in std_logic;
        Q : out std_logic
        );
end ffdSR;
```

```
architecture structural of ffdSR is

begin


Q <= '0' when RESET = '1' else '1' when SET = '1' else D when rising_edge(CLK) and EN = '1';

end structural;
```

A.14 counterSR_SRC

```
library ieee;
use ieee.std_logic_1164.all;

entity counterSR_SRC is
        port (
        CLK, clr: in std_logic;
        Q: out std_logic_vector (3 downto 0)
        );
end counterSR_SRC;

architecture structural of counterSR_SRC is

component adderC is
        port(
   A, B: in std_logic_vector(3 downto 0);
   Ci: in std_logic;
   Co: out std_logic;
   S: out std_logic_vector(3 downto 0)
   );
end component;

component regC is
        port(
        CLK : in std_logic;
        RESET : in std_logic;
        D : in std_logic_vector(3 downto 0);
        EN : in std_logic;
        Q : out std_logic_vector(3 downto 0)
        );
end component;

signal Adder_Reg: std_logic_vector (3 downto 0);
signal Reg_exit: std_logic_vector (3 downto 0);

begin

U0: adderC port map (A => Reg_exit, B => "0001", Ci => '0', Co => open, S => Adder_Reg);
U1: regC port map (CLK => CLK, RESET => clr, D => Adder_Reg, EN => '1', Q => Reg_exit);
Q <= Reg_exit;

end structural;
```

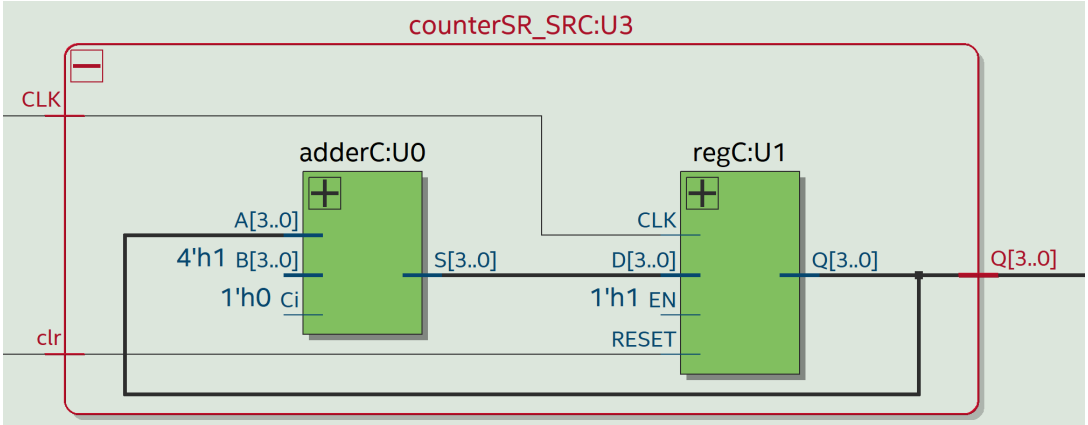

Figura 10: Netlist Viewer do *counterSR_SRC*

A.15 adderC

```
library ieee;
use ieee.std_logic_1164.all;

entity adderC is
        port(
        A, B: in std_logic_vector(3 downto 0);
        Ci: in std_logic;
        S: out std_logic_vector(3 downto 0);
        Co: out std_logic
        );
end adderC;

architecture structural of adderC is

component faC is
        port(
        A, B, Ci : in std_logic;
        S, Co: out std_logic
        );
end component;

signal carry: std_logic_vector(3 downto 1);

begin

U0: faC port map (A => A(0), B => B(0), Ci => Ci, S => S(0), Co => carry(1));
U1: faC port map (A => A(1), B => B(1), Ci => carry(1), S => S(1), Co => carry(2));
U2: faC port map (A => A(2), B => B(2), Ci => carry(2), S => S(2), Co => carry(3));
U3: faC port map (A => A(3), B => B(3), Ci => carry(3), S => S(3), Co => Co);

end structural;
```

A.16 faC

```
library ieee;
use ieee.std_logic_1164.all;

entity faC is
        port(
   A, B, Ci: in std_logic;
```

```
   S, Co: out std_logic
   );
end faC;

architecture structural of faC is

component haC is
    port(
    A, B: in std_logic;
    S, Co: out std_logic
    );
end component;

signal addAB: std_logic;
signal carry: std_logic_vector(1 downto 0);

begin

U1: haC port map (A => A, B => B, S => addAB, Co => carry(0));
U2: haC port map (A => addAB, B => Ci, S => S, Co => carry(1));

Co <= carry(0) or carry(1);


end structural;
```

A.17 haC

```
library ieee;
use ieee.std_logic_1164.all;

entity haC is
        port(
        A, B: in std_logic;
        S, Co: out std_logic
        );
end haC;

architecture structural of haC is
begin

S <= A xor B;
Co <= A and B;

end structural;
```

A.18 regC

```
library ieee;
use ieee.std_logic_1164.all;

entity regC is
        port(
        CLK : in std_logic;
        RESET : in std_logic;
        D : in std_logic_vector(3 downto 0);
        EN : in std_logic;
        Q : out std_logic_vector(3 downto 0)
        );
end regC;

architecture structural of regC is
```

```
component ffdC is
        port(
        CLK: in std_logic;
        RESET: in std_logic;
        SET: in std_logic;
        D: in std_logic;
        EN: in std_logic;
        Q: out std_logic
        );
end component;

begin

U0: ffdC port map(CLK => CLK, RESET => RESET, SET => '0', D => D(0), EN => EN, Q => Q(0));
U1: ffdC port map(CLK => CLK, RESET => RESET, SET => '0', D => D(1), EN => EN, Q => Q(1));
U2: ffdC port map(CLK => CLK, RESET => RESET, SET => '0', D => D(2), EN => EN, Q => Q(2));
U3: ffdC port map(CLK => CLK, RESET => RESET, SET => '0', D => D(3), EN => EN, Q => Q(3));

end structural;
```

A.19 ffdC

```
library ieee;
use ieee.std_logic_1164.all;

entity ffdC is
        port(
        CLK : in std_logic;
        RESET : in std_logic;
        SET : in std_logic;
        D : in std_logic;
        EN: in std_logic;
        Q : out std_logic
        );
end ffdC;

architecture structural of ffdC is

begin


Q <= '0' when RESET = '1' else '1' when SET = '1' else D when rising_edge(CLK) and EN = '1';

end structural;
```

A.20 comparatorSR_SRC

```
library ieee;
use ieee.std_logic_1164.all;

entity comparatorSR_SRC is
        port (
        A: in std_logic_vector (3 downto 0);
        B: in std_logic_vector (3 downto 0);
        TC: out std_logic
        );
end comparatorSR_SRC;

architecture structural of comparatorSR_SRC is
begin
```

```
TC <= ((A(0) xnor B(0)) and (A(1) xnor B(1)) and (A(2) xnor B(2)) and (A(3) xnor B(3)) );

end structural;
```

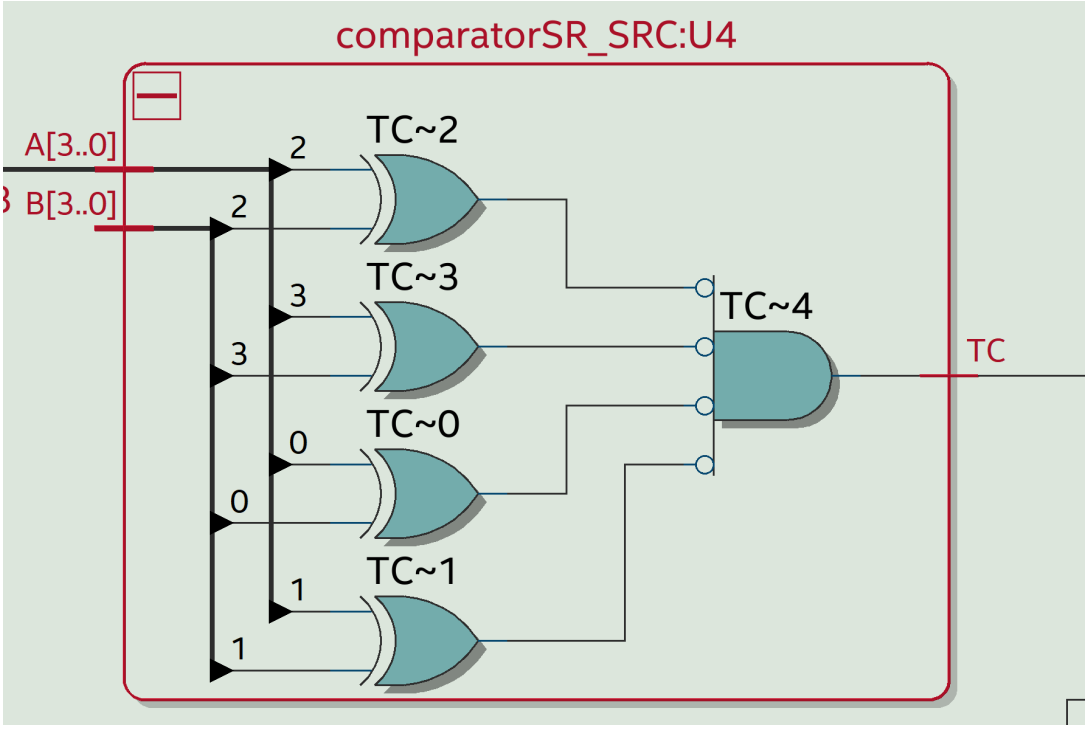

Figura 11: Netlist Viewer do *comparatorSR_SRC*

B Kotlin

B.1 RouletteDisplay

Algoritmo 1: RouletteDisplay

```
import isel.leic.utils.Time

object RouletteDisplay {
    const val UPDATE_DISPLAY = 0x06
    const val DISPLAY_ON = 0x07
    const val DISPLAY_OFF = 0x0F
    const val CMD_MASK = 0x07
    const val DATA_MASK = 0x1F

    private var startTime: Long = 0
    private var currentPosition: Int = 0
    private var isAnimating: Boolean = false
    private var lastUpdate: Long = 0
    val positions = listOf(0x11, 0x16, 0x15, 0x14, 0x13, 0x12)

    fun init() {
        SerialEmitter.send(SerialEmitter.Destination.ROULETTE, DISPLAY_ON and CMD_MASK, 8)
        SerialEmitter.send(SerialEmitter.Destination.ROULETTE, UPDATE_DISPLAY and CMD_MASK, 8)
    }

    fun animation() {
        if (!isAnimating) {
            startTime = Time.getTimeInMillis()
            currentPosition = 0
            isAnimating = true
        }

        if (isAnimating) {
            val currentTime = Time.getTimeInMillis()
            if (currentTime - startTime >= 5000L) {
                isAnimating = false
                return
            }

            if (currentTime - lastUpdate >= 15) {
                val currentSymbol = positions[currentPosition]
                setValue(currentSymbol)
                currentPosition = (currentPosition + 1) % positions.size
                lastUpdate = currentTime
            }
        }
    }

    // Exibe o mesmo s mbolo (value) em todos os d gitos do display
    fun setValue( value: Int ) {
        val values = IntArray(6) { value }

        for (i in 0..5) {
            val cmd = i and CMD_MASK
            val frame = (0 and DATA_MASK shl 3) or cmd
            SerialEmitter.send(SerialEmitter.Destination.ROULETTE, frame, 8)
        }

        for (i in 0..5) {
            val digit = values[i] // Envia o s mbolo inteiro para cada posi   o
            val cmd = i and CMD_MASK
            val frame = (digit and DATA_MASK shl 3) or cmd

            SerialEmitter.send(SerialEmitter.Destination.ROULETTE, frame, 8)
        }

        SerialEmitter.send(SerialEmitter.Destination.ROULETTE, UPDATE_DISPLAY, 8)
    }

    // Exibe um n mero inteiro, convertendo-o para d gitos individuais no display
    fun setNumber(number: Int) {
        val numberStr = number.toString()
```

```
        val values = IntArray(6) { 0x1F }

        for (i in numberStr.indices) {
            val pos = i
            values[pos] = numberStr[numberStr.length - 1 - i].digitToInt()
        }
        for (i in 0..5) {
            val cmd = i and CMD_MASK
            val frame = (0 and DATA_MASK shl 3) or cmd
            SerialEmitter.send(SerialEmitter.Destination.ROULETTE, frame, 8)
            Time.sleep(2)
        }

        for (i in 0..5) {
            val digit = values[i] and DATA_MASK
            val cmd = i and CMD_MASK
            val frame = (digit shl 3) or cmd
            SerialEmitter.send(SerialEmitter.Destination.ROULETTE, frame, 8)
            Time.sleep(2)
        }

        SerialEmitter.send(SerialEmitter.Destination.ROULETTE, UPDATE_DISPLAY, 8)
        Time.sleep(2)
    }

    fun off( value: Boolean ) {
        if (value) {
            SerialEmitter.send(SerialEmitter.Destination.ROULETTE, DISPLAY_OFF, 8)
            Time.sleep(2)
        } else {
            SerialEmitter.send(SerialEmitter.Destination.ROULETTE, DISPLAY_ON, 8)
            Time.sleep(2)
        }
    }

    // Exibe uma sequ ncia personalizada de s mbolos nos 6 d gitos do display
    fun setValues(values: IntArray) {
        for (i in 0..5) {
            val cmd = i and CMD_MASK
            val frame = (0 and DATA_MASK shl 3) or cmd
            SerialEmitter.send(SerialEmitter.Destination.ROULETTE, frame, 8)
            Time.sleep(2)
        }

        for (i in 0..5) {
            val digit = values[5 - i] and DATA_MASK // inverte a ordem
            val cmd = i and CMD_MASK
            val frame = (digit shl 3) or cmd
            SerialEmitter.send(SerialEmitter.Destination.ROULETTE, frame, 8)
            Time.sleep(2)
        }

        SerialEmitter.send(SerialEmitter.Destination.ROULETTE, UPDATE_DISPLAY, 8)
        Time.sleep(2)
    }

    fun clear(){
        val vazio = IntArray(6) { 0x1F }
        setValues(vazio)
        // ou setValue(0x1F)
    }

    fun mapCharToCode(ch: Char): Int = when (ch) {
        in '0'..'9' -> ch.digitToInt()
        'A' -> 0x0A
        'B' -> 0x0B
        'C' -> 0x0C
        'D' -> 0x0D
        else -> 0x1F
    }
}

fun main() {
    HAL.init()
```

```
    SerialEmitter.init()
    RouletteDisplay.init()

    val start = Time.getTimeInMillis()
    while (Time.getTimeInMillis() - start < 5000L) {
        RouletteDisplay.animation()
        Time.sleep(100)
    }
    RouletteDisplay.off(true)
    Time.sleep(1000)
    RouletteDisplay.off(false)
    RouletteDisplay.setValue(0x0B)
    Time.sleep(1000)
    RouletteDisplay.setNumber(100)
    Time.sleep(1000)
    RouletteDisplay.setValues(intArrayOf(0x1F, 0x1F, 1, 9, 0, 4))
}
```

B.2 SerialEmitter

Algoritmo 2: SerialEmitter

```
// Envia tramas para os diferentes modulos Serial Receiver.
object SerialEmitter {
    enum class Destination {LCD, ROULETTE}

    const val SDX_MASK = 0x01
    const val SCLK_MASK = 0x02
    const val LCD_SEL_MASK = 0x04
    const val P_MASK = 0x01
    const val RD_SEL_MASK = 0x08

    // Inicia a classe
    fun init () = HAL.setBits(LCD_SEL_MASK or RD_SEL_MASK)

    // Envia uma trama para o Serial Receiver
    // identificado o destino em     addr    ,
    // os bits de dados em     data
    // e em     size     o numero de bits a enviar.
    fun send(addr: Destination, data: Int, size: Int) {

        val destinationBit = if (addr == Destination.LCD) LCD_SEL_MASK else RD_SEL_MASK

        HAL.clrBits(destinationBit)

        var parity = 0

        for (i in 0 until size) {
            val bit = (data shr i) and P_MASK
            if (bit == 1) HAL.setBits(SDX_MASK) else HAL.clrBits(SDX_MASK)

            HAL.setBits(SCLK_MASK)
            HAL.clrBits(SCLK_MASK)

            parity = parity xor bit
        }

        // Envia o bit de paridade
        if (parity == 0) HAL.setBits(SDX_MASK) else HAL.clrBits(SDX_MASK)

        HAL.setBits(SCLK_MASK)
        HAL.clrBits(SCLK_MASK)

        HAL.setBits(destinationBit)
    }
}

fun main() {
    HAL.init()
    SerialEmitter.init()
    SerialEmitter.send(SerialEmitter.Destination.LCD, 0x15, 5)
}
```

B.3 HAL

Algoritmo 3: HAL

```
import isel.leic.UsbPort

object HAL {
    private var currentState = 0x00

    // Inicia o objeto
    fun init() = UsbPort.write(currentState)

    // Retorna 'true' se o bit definido pela mask esta com o valor logico '1' no UsbPort
    fun isBit(mask: Int): Boolean{
        return (UsbPort.read() and mask) != 0
    }

    // Retorna os valores dos bits representados por mask presentes no UsbPort
    fun readBits(mask: Int): Int{
        return UsbPort.read() and mask
    }

    // Escreve nos bits representados por mask os valores dos bits correspondentes em value
    fun writeBits(mask: Int, value: Int){
        clrBits(mask)
        setBits(value and mask)
    }

    // Coloca os bits representados por mask no valor logico '1'
    fun setBits(mask: Int){
        currentState = currentState or mask
        UsbPort.write(currentState)  // Atualiza o estado no hardware
    }

    // Coloca os bits representados por mask no valor logico '0'
    fun clrBits(mask: Int){
        currentState = currentState and mask.inv()
        UsbPort.write(currentState)  // Atualiza o estado no hardware
    }
}

fun main(){
    HAL.init() // Inicializa o HAL (estado inicial = 0)
    HAL.isBit(0x01) // Verifica se o bit 0 esta ativo
    HAL.readBits(0xFF) // Le o estado atual de todos os bits
    HAL.writeBits(0xF0 , 0xA0) // Escreve 1010_0000 nos bits 4 a 7 (com mascara 0xF0)
    HAL.setBits(0x0F) // Ativa os primeiros 4 bits (bits 0 a 3)
    HAL.clrBits(0x05) // Limpa os bits 0 e 2 (mascara 0000_0101)
    println("Bit␣1␣ativo?␣${HAL.isBit(0x02)}") // Exibe o estado final do bit 1
}
```

C Atribuição de Pinos

```
set_global_assignment -name BOARD "MAX␣10␣DE10␣-␣Lite"
set_global_assignment -name DEVICE 10M50DAF484C6GES
set_global_assignment -name FAMILY "MAX␣10"

#CLOCK
set_location_assignment PIN_P11 -to CLK

#RESET
set_location_assignment PIN_F15 -to RESET

#KEYPAD
set_location_assignment PIN_W5   -to KeyPadL[0]
set_location_assignment PIN_AA14 -to KeyPadL[1]
set_location_assignment PIN_W12  -to KeyPadL[2]
set_location_assignment PIN_AB12 -to KeyPadL[3]
set_location_assignment PIN_AB11 -to KeyPadC[0]
set_location_assignment PIN_AB10 -to KeyPadC[1]
set_location_assignment PIN_AA9  -to KeyPadC[2]
set_location_assignment PIN_AA8  -to KeyPadC[3]

#Dval e D
set_location_assignment PIN_A8  -to D[0]
set_location_assignment PIN_A9  -to D[1]
set_location_assignment PIN_A10 -to D[2]
set_location_assignment PIN_B10 -to D[3]
set_location_assignment PIN_D13 -to Dval

#LCD
set_location_assignment PIN_W11    -to DLCD[4]
set_location_assignment PIN_AA10   -to DLCD[5]
set_location_assignment PIN_Y8     -to DLCD[6]
set_location_assignment PIN_Y7     -to DLCD[7]
set_location_assignment PIN_W8     -to RS
set_location_assignment PIN_V5     -to E

#Display
#============================================================
# HEX0
#============================================================
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX0[0]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX0[1]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX0[2]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX0[3]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX0[4]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX0[5]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX0[6]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX0[7]
set_location_assignment PIN_C14 -to HEX0[0]
set_location_assignment PIN_E15 -to HEX0[1]
set_location_assignment PIN_C15 -to HEX0[2]
set_location_assignment PIN_C16 -to HEX0[3]
set_location_assignment PIN_E16 -to HEX0[4]
```

```
set_location_assignment PIN_D17 -to HEX0[5]
set_location_assignment PIN_C17 -to HEX0[6]
set_location_assignment PIN_D15 -to HEX0[7]

#============================================================
# HEX1
#============================================================
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX1[0]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX1[1]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX1[2]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX1[3]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX1[4]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX1[5]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX1[6]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX1[7]
set_location_assignment PIN_C18 -to HEX1[0]
set_location_assignment PIN_D18 -to HEX1[1]
set_location_assignment PIN_E18 -to HEX1[2]
set_location_assignment PIN_B16 -to HEX1[3]
set_location_assignment PIN_A17 -to HEX1[4]
set_location_assignment PIN_A18 -to HEX1[5]
set_location_assignment PIN_B17 -to HEX1[6]
set_location_assignment PIN_A16 -to HEX1[7]

#============================================================
# HEX2
#============================================================
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX2[0]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX2[1]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX2[2]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX2[3]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX2[4]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX2[5]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX2[6]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX2[7]
set_location_assignment PIN_B20 -to HEX2[0]
set_location_assignment PIN_A20 -to HEX2[1]
set_location_assignment PIN_B19 -to HEX2[2]
set_location_assignment PIN_A21 -to HEX2[3]
set_location_assignment PIN_B21 -to HEX2[4]
set_location_assignment PIN_C22 -to HEX2[5]
set_location_assignment PIN_B22 -to HEX2[6]
set_location_assignment PIN_A19 -to HEX2[7]

#============================================================
# HEX3
#============================================================
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX3[0]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX3[1]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX3[2]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX3[3]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX3[4]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX3[5]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX3[6]
```

```
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX3[7]
set_location_assignment PIN_F21 -to HEX3[0]
set_location_assignment PIN_E22 -to HEX3[1]
set_location_assignment PIN_E21 -to HEX3[2]
set_location_assignment PIN_C19 -to HEX3[3]
set_location_assignment PIN_C20 -to HEX3[4]
set_location_assignment PIN_D19 -to HEX3[5]
set_location_assignment PIN_E17 -to HEX3[6]
set_location_assignment PIN_D22 -to HEX3[7]

#============================================================
# HEX4
#============================================================
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX4[0]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX4[1]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX4[2]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX4[3]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX4[4]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX4[5]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX4[6]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX4[7]
set_location_assignment PIN_F18 -to HEX4[0]
set_location_assignment PIN_E20 -to HEX4[1]
set_location_assignment PIN_E19 -to HEX4[2]
set_location_assignment PIN_J18 -to HEX4[3]
set_location_assignment PIN_H19 -to HEX4[4]
set_location_assignment PIN_F19 -to HEX4[5]
set_location_assignment PIN_F20 -to HEX4[6]
set_location_assignment PIN_F17 -to HEX4[7]

#============================================================
# HEX5
#============================================================
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX5[0]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX5[1]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX5[2]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX5[3]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX5[4]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX5[5]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX5[6]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX5[7]
set_location_assignment PIN_J20 -to HEX5[0]
set_location_assignment PIN_K20 -to HEX5[1]
set_location_assignment PIN_L18 -to HEX5[2]
set_location_assignment PIN_N18 -to HEX5[3]
set_location_assignment PIN_M20 -to HEX5[4]
set_location_assignment PIN_N19 -to HEX5[5]
set_location_assignment PIN_N20 -to HEX5[6]
set_location_assignment PIN_L19 -to HEX5[7]
```

	tatu	From	To	Assignment Name	Value	Enabled	Entity	Comment	Tag
1	✔ ...		CLK	Location	PIN_P11	Yes			
2	✔ ...		R...T	Location	PIN_F15	Yes			
3	✔ ...		Ke...0]	Location	PIN_W5	Yes			
4	✔ ...		Ke...1]	Location	PI...14	Yes			
5	✔ ...		Ke...2]	Location	PI...12	Yes			
6	✔ ...		Ke...3]	Location	PI...12	Yes			
7	✔ ...		Ke...0]	Location	PI...11	Yes			
8	✔ ...		Ke...1]	Location	PI...10	Yes			
9	✔ ...		Ke...2]	Location	PI...A9	Yes			
10	✔ ...		Ke...3]	Location	PI...A8	Yes			
11	✔ ...		Dval	Location	PI...13	Yes			
12	✔ ...		D[0]	Location	PIN_A8	Yes			
13	✔ ...		D[1]	Location	PIN_A9	Yes			
14	✔ ...		D[2]	Location	PI...10	Yes			
15	✔ ...		D[3]	Location	PIN_B10	Yes			
16	✔ ...		DL...4]	Location	PI...11	Yes			
17	✔ ...		DL...5]	Location	PI...10	Yes			
18	✔ ...		DL...6]	Location	PIN_Y8	Yes			
19	✔ ...		DL...7]	Location	PIN_Y7	Yes			
20	✔ ...		RS	Location	PIN_W8	Yes			
21	✔ ...		E	Location	PIN_V5	Yes			
22	✔ ...		HE...0]	Location	PIN_C14	Yes			
23	✔ ...		HE...1]	Location	PIN_E15	Yes			
24	✔ ...		HE...2]	Location	PIN_C15	Yes			
25	✔ ...		HE...3]	Location	PIN_C16	Yes			

Figura 12: Assignment Editor - parte 1

26	✔ ...		HE...4]	Location	PIN_E16	Yes			
27	✔ ...		HE...5]	Location	PI...17	Yes			
28	✔ ...		HE...6]	Location	PIN_C17	Yes			
29	✔ ...		HE...7]	Location	PI...15	Yes			
30	✔ ...		HE...0]	Location	PIN_C18	Yes			
31	✔ ...		HE...1]	Location	PI...18	Yes			
32	✔ ...		HE...2]	Location	PIN_E18	Yes			
33	✔ ...		HE...3]	Location	PIN_B16	Yes			
34	✔ ...		HE...4]	Location	PI...17	Yes			
35	✔ ...		HE...5]	Location	PI...18	Yes			
36	✔ ...		HE...6]	Location	PIN_B17	Yes			
37	✔ ...		HE...7]	Location	PI...16	Yes			
38	✔ ...		HE...0]	Location	PIN_B20	Yes			
39	✔ ...		HE...1]	Location	PI...20	Yes			
40	✔ ...		HE...2]	Location	PIN_B19	Yes			
41	✔ ...		HE...3]	Location	PI...21	Yes			
42	✔ ...		HE...4]	Location	PIN_B21	Yes			
43	✔ ...		HE...5]	Location	PIN_C22	Yes			
44	✔ ...		HE...6]	Location	PIN_B22	Yes			
45	✔ ...		HE...7]	Location	PI...19	Yes			
46	✔ ...		HE...0]	Location	PIN_F21	Yes			
47	✔ ...		HE...1]	Location	PIN_E22	Yes			
48	✔ ...		HE...2]	Location	PIN_E21	Yes			
49	✔ ...		HE...3]	Location	PIN_C19	Yes			

Figura 13: Assignment Editor - parte 2

50	✔ ...		HE...4]	Location	PIN_C20	Yes			
51	✔ ...		HE...5]	Location	PI...19	Yes			
52	✔ ...		HE...6]	Location	PIN_E17	Yes			
53	✔ ...		HE...7]	Location	PI...22	Yes			
54	✔ ...		HE...0]	Location	PIN_F18	Yes			
55	✔ ...		HE...1]	Location	PIN_E20	Yes			
56	✔ ...		HE...2]	Location	PIN_E19	Yes			
57	✔ ...		HE...3]	Location	PIN_J18	Yes			
58	✔ ...		HE...4]	Location	PI...19	Yes			
59	✔ ...		HE...5]	Location	PIN_F19	Yes			
60	✔ ...		HE...6]	Location	PIN_F20	Yes			
61	✔ ...		HE...7]	Location	PIN_F17	Yes			
62	✔ ...		HE...0]	Location	PIN_J20	Yes			
63	✔ ...		HE...1]	Location	PIN_K20	Yes			
64	✔ ...		HE...2]	Location	PIN_L18	Yes			
65	✔ ...		HE...3]	Location	PI...18	Yes			
66	✔ ...		HE...4]	Location	PI...20	Yes			
67	✔ ...		HE...5]	Location	PI...19	Yes			
68	✔ ...		HE...6]	Location	PI...20	Yes			
69	✔ ...		HE...7]	Location	PIN_L19	Yes			
70		<<...>>	<<...>>	<<new>>					

Figura 14: Assignment Editor - parte 3